

Administración de Sistemas Operativos

UD – 1

Script Bash

Ejercicios







ÍNDICE

¿Qué es Bash? **3**

¿Por qué aprender scripting Bash en ASIR? **3**

Estructura básica de un script **4**

Variables de entorno comunes en administración *4*
Variables de usuario *4*
Parámetros posicionales *5*
Parámetros Especiales *5*
Comillas: simples, dobles y escape *5*

Redireccionamiento y Tuberías **6**

Redireccionamiento *6*
Tuberías *6*

Estructuras Condicionales **7**

Comando Test *7*
Operadores lógicos *7*
Estructura IF *8*
Estructura Case *8*

Estructuras de Repetición (Bucles) **9**

Bucle for *9*
Bucle While *9*
Control de bucles *9*

Funciones **10**

Definición y llamada *10*
Parámetros *10*
Retorno de valores *10*

Plantilla de script profesional **11**

Descripción o cabecera del script *11*
Funciones *11*
Zona de Validación *11*
Principal *12*

1. EJERCICIO 1

Crea un script **saludo.sh** que reciba un nombre como primer argumento. Si no se pasa ningún argumento, debe mostrar "*Hola, mundo*". Si se pasa un argumento, debe mostrar *"Hola, <nombre>"*. Verifica que el script tenga permisos de ejecución.

2. EJERCICIO 2

Escribe un script que reciba un número entero como argumento e indique si es par o impar. Valida que el argumento sea realmente un número.

3. EJERCICIO 3

Pide al usuario un número por teclado y muestra su tabla de multiplicar del 1 al 10 usando un bucle **for**.

4. EJERCICIO 4

Crea un script que reciba un número y haga una cuenta regresiva desde ese número hasta 0, imprimiendo cada valor. Al llegar a 0, imprime *"¡Despegue!"*.

5. EJERCICIO 5

Pide números al usuario de forma repetida dentro de un bucle **while**. El script debe ir acumulando la suma y mostrar el total parcial tras cada número introducido. El bucle termina cuando el usuario escribe la palabra **fin**. Al salir, muestra la suma total y cuántos números se introdujeron.

6. EJERCICIO 6

Define una función **factorial** que reciba un número y devuelva su factorial. El script debe pedir el número al usuario y mostrar el resultado. Valida que el número sea positivo.

7. EJERCICIO 7

Crea un script que pida al usuario una palabra y un archivo de texto. El script debe leer el archivo línea a línea con un bucle **while** **read**, contar cuántas líneas contienen esa palabra y mostrar el número de línea y el contenido de cada coincidencia. Si no hay coincidencias, indícalo.

8. EJERCICIO 8

Pide al usuario una ruta de archivo. Comprueba:

- Si existe.
- Si es un archivo regular (no directorio).
- Si tiene permiso de lectura.
  Muestra un mensaje distinto según cada caso.

9. EJERCICIO 9

Crea un script que muestre un menú con 4 opciones (**1**: *fecha*, **2**: *usuarios conectados*, **3**: *directorio actual*, **4**: *salir*) y ejecute la acción correspondiente usando case. Repite el menú hasta que el usuario elija salir.

10. EJERCICIO 10

Crea un script backup.sh que reciba dos argumentos: un directorio origen y un directorio destino. Debe:

- Validar que se pasen exactamente 2 argumentos.
- Comprobar que el origen existe y es un directorio.
- Crear el destino si no existe.
- Copiar el contenido del origen al destino añadiendo la fecha al nombre de una subcarpeta (ej: backup_2026-09-17).
- Mostrar un mensaje final con la ruta creada.